

Examen de Fundamentos de Ingeniería del Software

Publicación de notas: Jueves, 22 de mayo de 2025

Revisión: Lunes, 26 de mayo de 2025, 11:00

Es preciso identificarse mediante apellidos y nombre, así como el carné de la UPM (o en su defecto DNI o pasaporte). Cualquier dispositivo electrónico, incluidos teléfonos móviles, debe estar apagado y guardado. Es obligatorio entregar el examen, aunque no conteste a ninguna pregunta. No se puede salir del examen durante los primeros 20 minutos. Este examen tiene una duración de 60 minutos y consta de 15 preguntas. Existen diferentes versiones del examen.

Cada pregunta solo tiene una respuesta correcta y todas las preguntas tienen la misma puntuación. Cada respuesta correcta suma 1 punto y cada respuesta incorrecta resta 0,33 puntos. Las respuestas en blanco no suman ni restan.

La RESPUESTA DEFINITIVA SE DEBE RELLENAR EN LA PLANTILLA DE RESPUESTAS. Solo se tendrán en cuenta las respuestas especificadas en la plantilla, no se tendrán en cuenta las anotaciones o contestaciones en el resto del examen.

Ejemplos de relleno de DNI/NIE/PASSPORT

	7	0	9	8	7	2	4	0	1	1	4	2	5	2	3	4	8	0	0
0																			
1																			
2																			
3																			
4																			
5																			
6																			
7																			
8																			
9																			

	ES	4	2	5	2	3	4	8	0	0	4	4	2	5	2	3	4	8	0
0																			
1																			
2																			
3																			
4																			
5																			
6																			
7																			
8																			
9																			

Ejemplos de Respuesta

	4	4	2	5	2	3	4	8	0
0									
1									
2									
3									
4									
5									
6									
7									
8									
9									



Pregunta 1:

En la fase de diseño de una aplicación software para una tienda de comercio electrónico se ha añadido una nueva clase al diagrama de clases de diseño cuya responsabilidad es solicitar al usuario la información necesaria para dar de alta un nuevo producto. ¿De qué tipo es la nueva clase que se ha añadido al diagrama?

- (a) Clase del dominio.
- (b) Clase de interfaz de usuario.**
- (c) Clase del sistema.
- (d) Controlador.

Pregunta 2:

Para pasar del diseño A (mostrado a la izquierda) al diseño B (mostrado a la derecha), se ha aplicado:



- (a) Abstracción por especialización.**
- (b) Abstracción por descomposición.
- (c) Abstracción por agregación.
- (d) Ninguna de las otras respuestas es verdadera.

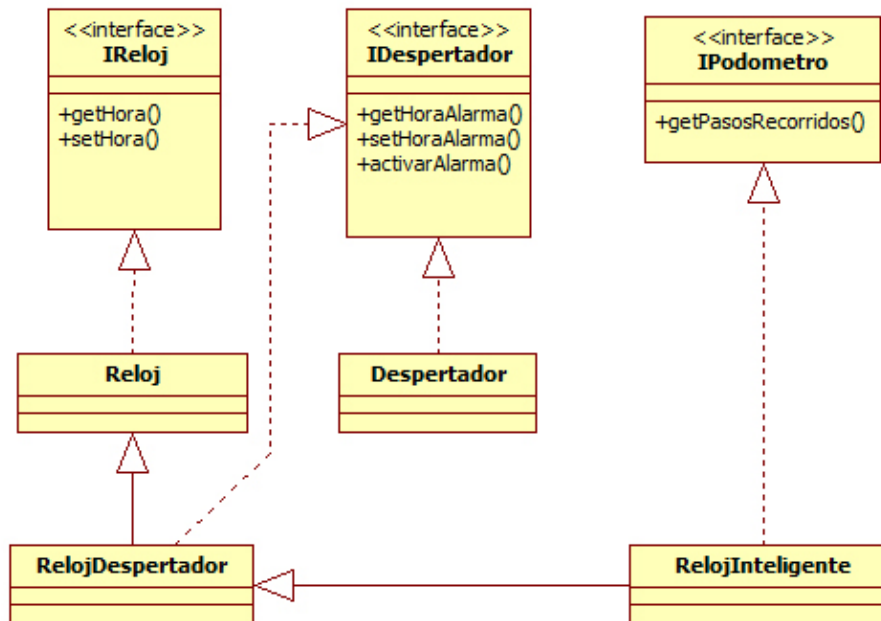
Pregunta 3:

Acaba de arrancar un proyecto en el que se debe desarrollar una aplicación software para gestionar bancos de cuestionarios. Esta aplicación debe permitir a profesores y alumnos ver los cuestionarios a través de un navegador web, así como descargar dichos cuestionarios en formato PDF. Además, la aplicación debe ofrecer una API a otros sistemas mediante la cual estos sistemas podrán obtener cuestionarios almacenados en el banco en diferentes formatos, incluyendo JSON y XML. En base a esta información, ¿cuál de los siguientes patrones arquitectónicos sería el más apropiado para el desarrollo de estas funcionalidades de la aplicación de gestión de bancos de cuestionarios?

- (a) State-Logic-Display.
- (b) Cliente-Servidor.
- (c) Sense-Compute-Control.
- (d) MVC.**

Pregunta 4:

Dado el siguiente fragmento de un diagrama de clases de diseño y considerando que se ha aplicado el principio segregación de interfaces, podemos afirmar que:



- (a) Se ha aplicado correctamente el principio de segregación de interfaces.
- (b) No se ha aplicado correctamente el principio de segregación de interfaces porque ninguna clase implementa múltiples interfaces.
- (c) No se ha aplicado correctamente el principio de segregación de interfaces porque hay interfaces pesadas, es decir, con más de dos métodos.
- (d) No se ha aplicado correctamente el principio de segregación de interfaces porque la clase “RelojInteligente” tiene múltiples responsabilidades.

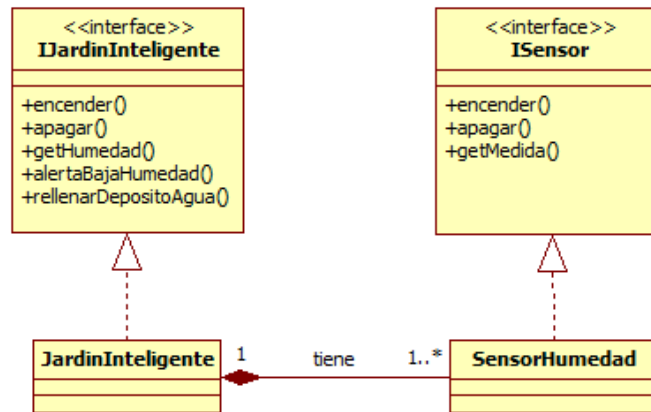
Pregunta 5:

Se pretenden realizar pruebas de caja negra utilizando tanto la técnica de particiones de equivalencia como la de análisis de valores límite para probar una función que recibe como entrada un número que puede ir de 1 a 100. ¿Qué casos de prueba se podrían definir para realizar estas pruebas?

- (a) CP1 (0), CP2 (1), CP3 (100), CP4 (101).
- (b) CP1 (-10), CP2 (50), CP3 (150).
- (c) CP1 (-10), CP2 (0), CP3 (1), CP4 (100), CP5 (101), CP6 (200).
- (d) CP1 (-10), CP2 (1), CP3 (100), CP4 (200).

Pregunta 6:

Dado el siguiente diagrama de clases de diseño y considerando que se ha aplicado el principio de inversión de dependencias, podemos afirmar que:



- (a) Se ha aplicado correctamente el principio de inversión de dependencias porque las clases “JardinInteligente” y “SensorHumedad” están totalmente desacopladas gracias al uso de interfaces.
- (b) Se ha aplicado correctamente el principio de inversión de dependencias porque la clase “SensorHumedad”, mediante la interfaz “ISensor”, permite a otros objetos que no sean de la clase “JardinInteligente” hacer uso de ella.
- (c) No se ha aplicado correctamente el principio de inversión de dependencias porque la relación “tiene” entre las clases “JardinInteligente” y “SensorHumedad” debería tener el sentido contrario.
- (d) No se ha aplicado correctamente el principio de inversión de dependencias porque existe un alto acoplamiento entre las clases “JardinInteligente” y “SensorHumedad”.

Pregunta 7:

Se pretenden realizar pruebas de caja negra empleando la técnica de particiones de equivalencia para probar el siguiente método:

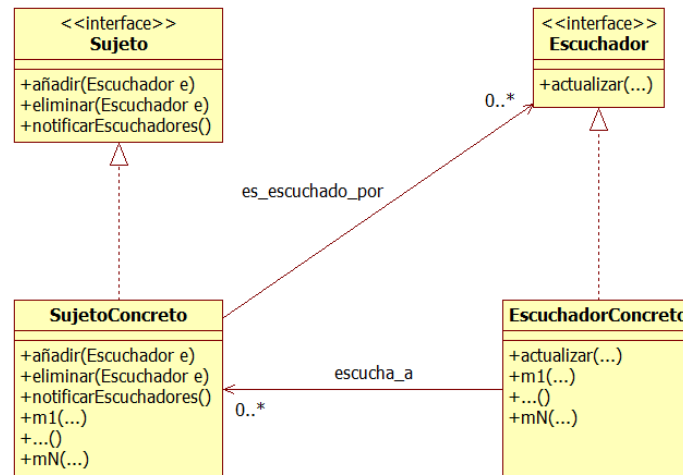
```
public boolean contenidoAccesible(String tipoContenido, String planSuscripcion){...}
```

Teniendo en cuenta que el valor de “tipoContenido” debería ser “Gratis”, “Freemium” o “Premium” y que el valor de “planSuscripcion” debería ser “Básico”, “Estándar” o “Pro”, se puede afirmar que:

- (a) Cada parámetro da lugar a 2 clases de equivalencia.
- (b) Habría que definir un total de 5 casos de prueba.
- (c) El caso de prueba CP(“Película”, “Mega”) permitiría probar de manera adecuada dos clases de equivalencia.
- (d) Ninguna de las otras respuestas es verdadera.

Pregunta 8:

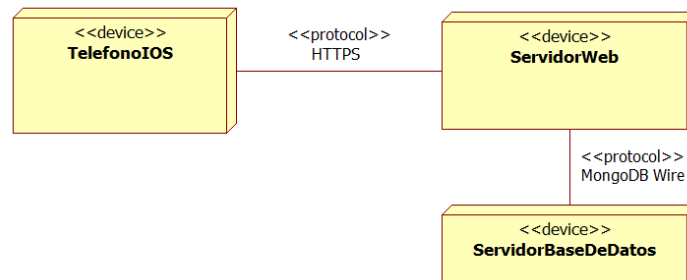
¿A qué patrón de diseño corresponde el siguiente diagrama de clases?



- (a) Command.
- (b) Proxy.
- (c) Observer.
- (d) Adapter.

Pregunta 9:

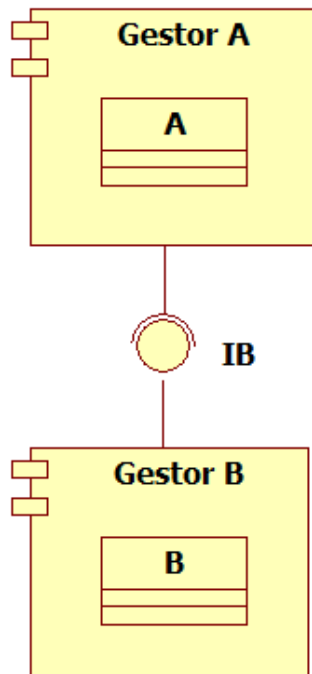
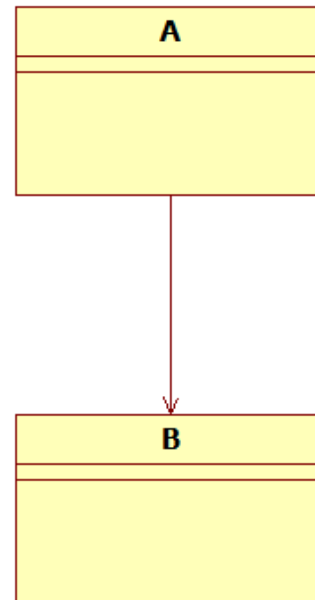
La siguiente figura muestra:



- (a) Un diagrama de clases con 2 interfaces.
- (b) Un diagrama de componentes con 3 nodos.
- (c) Un diagrama de despliegue con 3 componentes.
- (d) Ninguna de las otras respuestas es verdadera.

Pregunta 10:

Los diagramas A y B representan el diseño de un software desde dos perspectivas diferentes. ¿Es coherente el diagrama B con el diagrama A?

Diagrama A**Diagrama B**

- (a) Sí.
- (b) No, porque la clase A se relaciona directamente con la clase B en lugar de relacionarse con una interfaz implementada por la clase B.
- (c) No, porque la clase A se relaciona directamente con la clase B en lugar de relacionarse la clase B con una interfaz implementada por la clase A.
- (d) No, porque la relación entre las clases A y B no es una agregación.

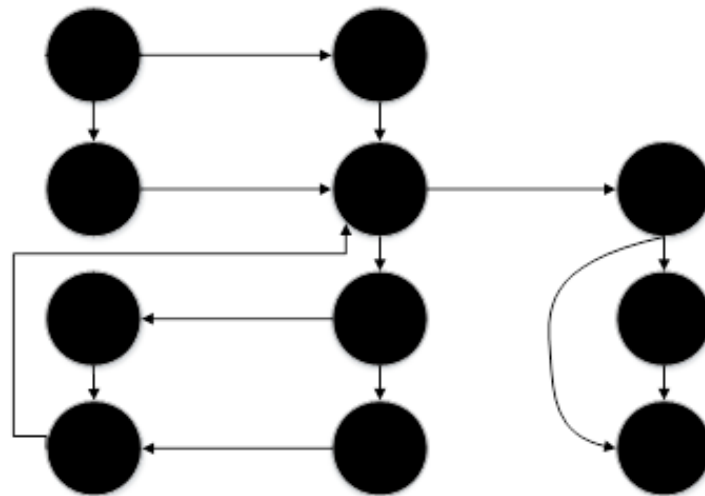
Pregunta 11:

Indique con cuál de los siguientes términos encaja mejor la siguiente definición: “Pruebas realizadas para determinar que al remplazar un componente por otro en un sistema, este, no ha afectado negativamente a la funcionalidad, fiabilidad o rendimiento y no ha introducido defectos adicionales”.

- (a) Pruebas unitarias.
- (b) Pruebas de seguridad.
- (c) Pruebas de regresión.
- (d) Pruebas del sistema.

Pregunta 12:

¿Cuál es la complejidad ciclomática del siguiente grafo de flujo construido mediante la prueba de la ruta base?



Pregunta 14:

Indique con cuál de los siguientes términos encaja mejor la siguiente definición: “Manera y grado en que las tareas realizadas por un único módulo de software están relacionadas entre sí”.

- (a) Acoplamiento.
- (b) **Cohesión.**
- (c) Modularidad.
- (d) Abstracción.

Pregunta 15:

Un programador ha definido una serie de casos de prueba aplicando la técnica de particiones de equivalencia para probar el método “setId” de la clase “Usuario”. Posteriormente, el programador ha implementado con JUnit todos los casos de prueba definidos. A continuación, se muestra el código de la clase “Usuario” (a la izquierda) y el de la implementación de estos casos de prueba (a la derecha):

```
public class Usuario {  
    private int id;  
    public Usuario(int id) throws Exception {  
        setId(id);  
    }  
    public int getId() {  
        return id;  
    }  
    public void setId(int id) throws Exception {  
        if (id < 1) {  
            throw new IllegalArgumentException();  
        }  
        if (id > 999) {  
            throw new IllegalArgumentException();  
        }  
        this.id = id;  
    }  
}
```

```
public class UsuarioTest {  
    Usuario u;  
    @Before  
    public void setUp() throws Exception {  
        u = new Usuario(1);  
    }  
    @Test  
    public void test1() throws Exception {  
        u.setId(42);  
        assertEquals(u.getId(), 42);  
    }  
    @Test (expected = Exception.class)  
    public void test2() throws Exception {  
        u.setId(1000);  
    }  
}
```

- (a) La prueba definida por el método “test1” permite probar de manera adecuada una clase de equivalencia inválida.
- (b) La implementación del método “test2” es errónea, ya que la prueba que define este método no permite probar de manera adecuada ninguna clase de equivalencia.
- (c) Las pruebas definidas permiten probar todas las clases de equivalencia.
- (d) **Ninguna de las otras respuestas es verdadera.**